

Master Header

```
#include <bits/stdc++.h>
using namespace std;
typedef long long int lli;
#define endl "\n"
#define INF 1e18
void fast() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
}
```

Binary_Index_Tree1.cpp

```
lli getSum(vector<lli>& BITree, lli ix) {
    lli sum = 0;
    ix++;
    while (ix > 0) {
        sum += BITree[ix];
        ix -= ix & (-ix);
    }
    return sum;
}
void updateBIT(vector<lli>& BITree, lli n, lli ix, lli val) {
    ix++;
    while (ix <= n) {
        BITree[ix] += val;
        ix += ix & (-ix);
    }
}
vector<lli> constructBITree(vector<lli>& arr, lli n) {
    vector<lli> BITree(n + 1, 0);
    for (lli i = 0; i < n; i++) updateBIT(BITree, n, i, arr[i]);
    return BITree;
}
```

binarySearch.cpp

```
lli binary_search(vector<lli> vec, lli lo, lli hi, lli x) {
    while (lo <= hi) {
        lli mid = lo + (hi - lo) / 2;
        if (vec[mid] == x)
            return mid;
        else if (vec[mid] < x)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    return -1;
}
lli lower_bound(vector<lli>& v, lli e) {
    lli lo = 0, hi = v.size() - 1;
    lli mid;
    while (lo <= hi) {
        lli mid = lo + (hi - lo) / 2;
        if (v[mid] < e)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    if (v[lo] >= e)
        return lo;
    if (v[hi] >= e)
        return hi;
    return -1;
}
lli upper_bound(vector<lli>& v, lli e) {
    lli lo = 0, hi = v.size() - 1;
    lli mid;
    while (lo <= hi) {
        lli mid = lo + (hi - lo) / 2;
        if (v[mid] <= e)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    if (v[lo] >= e)
        return lo;
    if (v[hi] >= e)
        return hi;
    return -1;
}
```

binomialCoefficients.cpp

```
const lli N = 1e6 + 10;
const lli MOD = 1e9 + 7;
lli factorial[N], inverseFactorial[N], modInverse[N];
void preCalculate_nCr() {
    factorial[0] = 1;
    for (lli i = 1; i < N; i++)
        factorial[i] = (1LL * i * factorial[i - 1] % MOD);
    modInverse[1] = 1;
    for (lli i = 2; i < N; i++) {
        modInverse[i] = (-(1LL * (MOD / i))) *
            modInverse[MOD % i] % MOD;
        modInverse[i] = (modInverse[i] + MOD) % MOD;
    }
    inverseFactorial[0] = 1;
    for (lli i = 1; i < N; i++)
        inverseFactorial[i] =
            (1LL * modInverse[i] * inverseFactorial[i - 1]
            % MOD);
}
lli nCr(lli n, lli r) {
    if (n < r || n < 0 || r < 0)
        return 0;
    else
        return (1LL * factorial[n] * inverseFactorial[n - r] % MOD *
            inverseFactorial[r] % MOD);
}
lli nPr(lli n, lli r) {
    if (n < r || n < 0 || r < 0)
        return 0;
    return (1LL * nCr(n, r) * factorial[r] % MOD);
}
```

BiPartiteGraph.cpp

```
const lli N = 1e7 + 10;
vector<bool> visited(N);
vector<lli> color(N);
bool ok;
void DFS(lli v, vector<vector<lli>>& graph) {
    visited[v] = true;
    // cout << v << " ";
    for (auto next = graph[v].begin(); next != graph[v].end(); ++next) {
        if (!visited[*next]) {
            color[*next] = color[v] ^ 1;
            DFS(*next, graph);
        } else {
            if (color[*next] == color[v])
                ok = false;
        }
    }
}
void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> graph(n + 1);
    while (m--) {
        lli u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    ok = true;
    lli start;
    cin >> start;
    DFS(start, graph);
    for (lli i = 1; i <= n; i++) {
        if (!visited[i])
            DFS(i, graph); // if the graph is not
    }
    if (ok)
        cout << "YES" << endl;
    else
        cout << "NO" << endl;
}
```

Bitwise_Operator_Properties.cpp

```
/*
Some properties of bitwise operations:
a|b = a^b + a&b
a^(a&b) = (a|b)^b
b^(a&b) = (a|b)^a
(a&b)^(a|b) = a^b
*/
```

```

Addition:
a+b = a|b + a&b
a+b = a^b + 2(a&b)
Subtraction:
a-b = (a^b)&b - ((a|b)^a)
a-b = ((a|b)^b) - ((a|b)^a)
a-b = (a^b)&b - (b^a&b)
a-b = ((a|b)^b) - (b^a&b)
*/

```

Bitwise_Tricks.cpp

```

// check LSB to see if x is odd or even
bool isEven(lli x) {
    if (x & 1)
        return false; // odd
    else
        return true; // even
}
// check if x is a power of 2
bool powerof2(lli x) {
    if ((x & (x - 1)) == 0)
        return true;
    else
        return false;
}
// check if the k-th bit is set
lli checkKthBit(lli x, lli k) { return x & (1 << k); }
// toggle the k-th bit
lli toggleKthBit(lli x, lli k) { return x ^ (1 << k); }
// set the k-th bit
lli setKthBit(lli x, lli k) { return x | (1 << k); }
// unset the k-th bit
lli unsetKthBit(lli x, lli k) { return x & ~(1 << k); }
// multiply x by 2^k
lli multiplyBy2powk(lli x, lli k) {
    return (x << k); // left shift
}
// divide x by 2^k
lli divideBy2powk(lli x, lli k) {
    return (x >> k); // right shift
}
// Number of bits to flip to convert x to y
lli bitFlip(lli x, lli y) { return __builtin_popcountll(x ^ y); }
// swap two numbers using XOR
void swap(lli &x, lli &y) {
    x = x ^ y;
    y = x ^ y;
    x = x ^ y;
}
// count the number of set bits
lli noOfSetBits(lli x) { return __builtin_popcountll(x); }
void soln() {
    lli x = 10, y = 20, k = 2;
    cout << "Is " << x << " even? " << isEven(x) << endl;
    cout << "Is " << x << " a power of 2? " << powerof2(x) << endl;
    cout << "Check " << k << "-th bit of " << x << ": " <<
        checkKthBit(x, k) << endl;
    cout << "Toggle " << k << "-th bit of " << x << ": " <<
        toggleKthBit(x, k) << endl;
    cout << "Set " << k << "-th bit of " << x << ": " <<
        setKthBit(x, k) << endl;
    cout << "Unset " << k << "-th bit of " << x << ": " <<
        unsetKthBit(x, k) << endl;
    cout << "Multiply " << x << " by 2^" << k << ": " <<
        multiplyBy2powk(x, k) << endl;
    cout << "Divide " << x << " by 2^" << k << ": " <<
        divideBy2powk(x, k) << endl;
    cout << "Number of set bits in " << x << ": " <<
        noOfSetBits(x) << endl;
    cout << "Number of bits to flip to convert " << x << "
        to " << y << ": " <<
        bitFlip(x, y) << endl;
    cout << "Before swap: x = " << x << ", y = " << y <<
        endl;
    swap(x, y);
    cout << "After swap: x = " << x << ", y = " << y <<
        endl;
}

```

BFS2.cpp

```

const lli N = 1e5 + 10;
vector<bool> visited(N, false);
vector<lli> parent(N, -1);
vector<lli> dist(N, -1);
vector<vector<lli>> graph(N);
void printPath(lli node) {
    vector<lli> path;
    while (node != -1) {
        path.push_back(node);
        node = parent[node];
    }
    reverse(path.begin(), path.end());
    for (lli i = 0; i < (lli)path.size(); i++) {
        if (i > 0)
            cout << "->";
        cout << path[i];
    }
    cout << endl;
}
void BFS(lli start, vector<pair<lli, lli>> &BFSTree) {
    queue<lli> q;
    q.push(start);
    visited[start] = true;
    dist[start] = 0;
    cout << "BFS Order: ";
    while (!q.empty()) {
        lli v = q.front();
        cout << v << " ";
        q.pop();
        for (lli neighbor : graph[v]) {
            if (!visited[neighbor]) {
                q.push(neighbor);
                visited[neighbor] = true;
                parent[neighbor] = v;
                dist[neighbor] = dist[v] + 1;
                BFSTree.push_back({v, neighbor});
            }
        }
    }
    cout << endl;
}

```

DFS2.cpp

```

const lli N = 2e5 + 10;
vector<vector<lli>> graph(N);
vector<bool> visited(N, false);
vector<lli> discovery(N, 0), finishing(N, 0), parent(N, -1);
map<pair<lli, lli>, string> edgeType;
lli tct = 0;
void classify_edge(lli u, lli v) {
    if (!visited[v]) {
        edgeType[{u, v}] = "Tree Edge";
    } else if (finishing[v] == 0) {
        edgeType[{u, v}] = "Back Edge";
    } else if (discovery[u] < discovery[v]) {
        edgeType[{u, v}] = "Forward Edge";
    } else {
        edgeType[{u, v}] = "Cross Edge";
    }
}
void DFS(lli v, vector<pair<lli, lli>>& edges) {
    visited[v] = true;
    discovery[v] = ++tct;
    cout << v << " ";
    for (lli neighbor : graph[v]) {
        classify_edge(v, neighbor);
        if (!visited[neighbor]) {
            parent[neighbor] = v;
            DFS(neighbor, edges);
        }
    }
    finishing[v] = ++tct;
}
void soln() {
    lli n, e;
    cin >> n >> e;
    for (lli i = 1; i <= n; i++) {
        graph[i].clear();
        visited[i] = false;
        discovery[i] = finishing[i] = 0;
        parent[i] = -1;
    }
    edgeType.clear();
    vector<pair<lli, lli>> edges;
    for (lli i = 0; i < e; i++) {

```

```

    lli u, v;
    cin >> u >> v;
    graph[u].push_back(v);
    edges.emplace_back(u, v);
}
cout << "Adjacency List:" << endl;
for (lli i = 1; i <= n; i++) {
    cout << i << ": ";
    for (lli j : graph[i]) {
        cout << j << " ";
    }
    cout << endl;
}
cout << "DFS order: ";
for (lli i = 1; i <= n; i++) {
    if (!visited[i]) {
        DFS(i, edges);
    }
}
cout << endl;
cout << "Timestamps of Vertex(discovery/finishing):"
<< endl;
for (lli i = 1; i <= n; i++) {
    cout << i << "(" << discovery[i] << "/" <<
finishing[i] << ")" << endl;
}
cout << "Edge classification:" << endl;
for (auto& [u, v] : edges) {
    cout << u << " " << v << ": " << edgeType[{u, v}]
<< endl;
}
}

```

BellmanFord.cpp

```

vector<tuple<lli, lli, lli>> edges; // {u, v, weight}
vector<lli> dist(N + 1, INF);
bool bellmanFord(lli n, lli start) {
    dist[start] = 0;
    for (lli i = 1; i < n; i++) {
        for (auto& [u, v, w] : edges) {
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    // Detect negative cycle
    for (auto& [u, v, w] : edges) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            return false; // Negative cycle found
        }
    }
    return true;
}

```

dijkstraAlgorithmPath.cpp

```

const lli N = 2e6 + 10;
vector<vector<pair<lli, lli>>> graph(N); // {node, weight}
vector<lli> dist(N + 1, INF);
vector<lli> parent(N + 1, -1);
priority_queue<pair<lli, lli>, vector<pair<lli, lli>>, greater<pair<lli, lli>>>
pq;
void dijkstra(lli start) {
    dist[start] = 0;
    parent[start] = -1;
    pq.push({0, start});
    while (!pq.empty()) {
        lli d = pq.top().first;
        lli u = pq.top().second;
        pq.pop();
        if (d > dist[u])
            continue;
        for (auto& [v, w] : graph[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
}
vector<lli> getPath(lli target) {
    vector<lli> path;
    while (target != -1) {

```

```

        path.push_back(target);
        target = parent[target];
    }
    reverse(path.begin(), path.end());
    return path;
}
void soln() {
    lli n, m;
    cin >> n >> m;
    for (lli i = 0; i < m; i++) {
        lli u, v, w;
        cin >> u >> v >> w;
        graph[u].push_back({v, w});
        graph[v].push_back({u, w});
    }
    lli source = 1, target = n;
    // cin >> source >> target;
    dijkstra(source);
    if (dist[target] == INF) {
        // cout << "No path from " << source << " to " <<
target << endl;
        cout << -1 << endl;
    } else {
        // cout << "Shortest distance from " << source <<
" to " << target << "
// is: " << dist[target] << endl;
        vector<lli> path = getPath(target);
        // cout << "Path: ";
        for (lli node : path) {
            cout << node << " ";
        }
        cout << endl;
    }
}
}

```

floydWarshall.cpp

```

void floydWarshall(vector<vector<lli>> &dist, lli n) {
    for (lli k = 1; k <= n; k++) {
        for (lli i = 1; i <= n; i++) {
            for (lli j = 1; j <= n; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i]
[k] + dist[k][j]);
            }
        }
    }
}
void soln() {
    lli n, m, q;
    cin >> n >> m >> q;
    vector<vector<lli>> dist(n + 1, vector<lli>(n + 1,
INF));
    for (lli i = 0; i < m; i++) {
        lli u, v, w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w);
        dist[v][u] = min(dist[v][u], w);
    }
    for (lli i = 1; i <= n; i++) dist[i][i] = 0;
    floydWarshall(dist, n);
    while (q--) {
        lli s, e;
        cin >> s >> e;
        if (dist[s][e] == INF)
            cout << -1 << endl;
        else
            cout << dist[s][e] << endl;
    }
}

```

pathFindinginGrid.cpp

```

/* A (n*m) grid where you can traverse through if there
are 0's and 1's
are blocks, your job is to find if you can reach from top
left to bottom right
*/
vector<lli> dx = {1, -1, 0, 0};
vector<lli> dy = {0, 0, 1, -1};
bool canReach(vector<vector<lli>> &grid) {
    lli n = grid.size(), m = grid[0].size();
    if (grid[0][0] == 1 || grid[n - 1][m - 1] == 1)
        return false;
    queue<pair<lli, lli>> q;
    q.push({0, 0});

```

```

vector<vector<bool>> visited(n, vector<bool>(m,
false));
visited[0][0] = true;
while (!q.empty()) {
    auto [x, y] = q.front();
    q.pop();
    if (x == n - 1 && y == m - 1)
        return true;
    for (lli d = 0; d < 4; d++) {
        lli nx = x + dx[d], ny = y + dy[d];
        if (nx >= 0 && ny >= 0 && nx < n && ny < m &&
grid[nx][ny] == 0 &&
        !visited[nx][ny]) {
            visited[nx][ny] = true;
            q.push({nx, ny});
        }
    }
}
return false;
}

void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> grid(n, vector<lli>(m));
    for (lli i = 0; i < n; i++)
        for (lli j = 0; j < m; j++) cin >> grid[i][j];
    cout << (canReach(grid) ? "YES" : "NO") << endl;
}

```

kruskalAlgorithm.cpp (obidit)

```

const lli N = 2e6 + 10;
lli parent[N];
lli rank_[N];
void make(lli v) {
    parent[v] = v;
    rank_[v] = 0;
}
lli find(lli v) {
    if (v == parent[v])
        return v;
    return parent[v] = find(parent[v]); // path
compression
}
void Union(lli a, lli b) {
    a = find(a);
    b = find(b);
    if (a != b) {
        if (rank_[a] < rank_[b]) {
            parent[a] = b;
        } else {
            parent[b] = a;
            if (rank_[a] == rank_[b]) {
                rank_[a]++;
            }
        }
    }
}
bool comparator(const vector<lli>& a, const vector<lli>&
b) {
    return a[2] < b[2]; // ascending by weight
}
lli KruskalsMST(lli v, vector<vector<lli>>& edges) {
    sort(edges.begin(), edges.end(), comparator);
    lli cost = 0, ct = 0;
    for (auto& e : edges) {
        lli u = e[0], vv = e[1], w = e[2];
        if (find(u) != find(vv)) {
            Union(u, vv);
            cost += w;
            ct++;
            if (ct == v - 1)
                break;
        }
    }
    if (ct < v - 1)
        return -1; // Graph is disconnected
    return cost;
}

```

Kruskal (rafio bhai).cpp

```

using LL = long long;
using PLL = pair<LL, LL>;
using TLLL = tuple<LL, LL, LL>;

using Edge = TLLL; // node1, node2, weight
using Graph = vector<Edge>;

```

```

struct DSU
{
    vector<int> parent, depth;

    DSU(int n) : parent(vector<int>(n)),
depth(vector<int>(n))
    {
        int i;
        for (i = 0; i < n; i++)
            makeSet(i);
    }

    void makeSet(int v)
    {
        parent[v] = v;
        depth[v] = 0;
    }

    int getRoot(int v)
    {
        int pr = parent[v];
        if (pr == v)
            return pr;
        return parent[v] = getRoot(pr);
    }

    void unionSets(int a, int b)
    {
        a = getRoot(a);
        b = getRoot(b);

        if (a == b)
            return;
        if (depth[a] < depth[b])
            swap(a, b);

        parent[b] = a;
        if (depth[a] == depth[b])
            depth[a]++;
    }

    bool isSameRoot(int a, int b)
    {
        return getRoot(a) == getRoot(b);
    }
};

bool comp(Edge e1, Edge e2)
{
    auto [u1, v1, w1] = e1;
    auto [u2, v2, w2] = e2;
    return w1 < w2;
}

void mstKruskal(int n, Graph &graph, Graph &tree)
{
    sort(graph.begin(), graph.end(), comp);
    DSU dsu(n);
    for (auto [u, v, w] : graph)
    {
        if (dsu.isSameRoot(u, v))
            continue;

        dsu.unionSets(u, v);
        tree.push_back({u, v, w});
    }
}

```

primsAlgorithm.cpp (obidit)

```

const lli N = 2e6 + 10;
vector<vector<pair<lli, lli>>> adj(N);
vector<bool> visited(N, false);
priority_queue<pair<lli, lli>, vector<pair<lli, lli>>,
greater<pair<lli, lli>>> pq;
lli Prims(lli vertices, lli edges, vector<vector<lli>>&
edgeList) {
    for (lli i = 0; i < vertices; i++) {
        adj[i].clear();
        visited[i] = false;
    }
    while (!pq.empty())
        pq.pop();
    for (lli i = 0; i < edges; i++) {
        lli u = edgeList[i][0] - 1;
        lli v = edgeList[i][1] - 1;
        lli w = edgeList[i][2];
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
}

```

```

}
lli ans = 0;
lli visCt = 0;
pq.push({0, 0});
while (!pq.empty()) {
    auto [cost, u] = pq.top();
    pq.pop();
    if (visited[u])
        continue;
    visited[u] = true;
    ans += cost;
    visCt++;
    for (auto [neighbor, weight] : adj[u]) {
        if (!visited[neighbor]) {
            pq.push({weight, neighbor});
        }
    }
}
if (visCt != vertices) {
    cout << "IMPOSSIBLE" << endl;
    return -1;
}
return ans;
}

```

Prims(rafio bhai).cpp

```

using LL = long long;
using PLL = pair<LL, LL>;
using TLLL = tuple<LL, LL, LL>;
using Neighbor = PLL; // node, weight
using Graph = vector<vector<Neighbor>>;

void mstPrim(Graph& graph, vector<int>& dist, vector<int>& parent)
{
    int n = graph.size();
    priority_queue<TLLL, vector<TLLL>, greater<TLLL>>
    pq; // weight, node, parent
    pq.push({0, 0, -1});
    while (!pq.empty())
    {
        auto [d1, x, p] = pq.top();
        pq.pop();
        if (dist[x] != -1) continue;
        dist[x] = d1;
        parent[x] = p;
        for (auto [y, d2] : graph[x])
        {
            if (dist[y] != -1) continue;
            pq.push({d2, y, x});
        }
    }
}

```

minQueriesSparseTable.cpp

```

const lli N = 2e6 + 10;
const lli M = 21; // log2(N) ≈ 21
vector<vector<lli>> table(N, vector<lli>(M, 0));
void buildSparseTable(vector<lli>& arr, lli n) {
    for (lli i = 0; i < n; i++) table[i][0] = arr[i];
    for (lli j = 1; (1 << j) <= n; j++) {
        for (lli i = 0; (i + (1 << j) - 1) < n; i++) {
            table[i][j] =
                min(table[i][j - 1], table[i + (1 << (j - 1))][j - 1]);
        }
    }
}
void soln() {
    lli n, q;
    cin >> n >> q;
    vector<lli> arr(n);
    for (lli i = 0; i < n; i++) cin >> arr[i];
    buildSparseTable(arr, n);
    while (q--) {
        lli l, r;
        cin >> l >> r;
        l--, r--; // Convert to 0-based indexing
        lli p = (lli)log2(r - l + 1);
        lli k = (1 << p);
        lli ans = min(table[l][p], table[r - k + 1][p]);
        cout << ans << endl;
    }
}

```

String_Matching_Hashing.cpp

```

const lli BASE = 99991; // Large Prime Number
const lli M = 1e9 + 9;

// Implementation of Rabin-Karp Algorithm
lli RK(const string& text, const string& pattern) {
    lli T = text.size();
    lli P = pattern.size();
    // Precomputing powers of BASE modulo M
    vector<lli> powers(max(T, P));
    powers[0] = 1;
    for (lli i = 1; i < (lli)powers.size(); i++) {
        powers[i] = (powers[i - 1] * BASE) % M;
    }
    // for (auto px : powers) {
    //     cout << px << " ";
    // }
    // cout << endl;
    // Computing prefix hashes for the text
    vector<lli> hash(T + 1, 0);
    for (lli i = 0; i < T; i++) {
        hash[i + 1] = (hash[i] + (text[i] - 'a' + 1) *
        powers[i]) % M;
    }
    // for (auto hh : hash) {
    //     cout << hh << " ";
    // }
    // cout << endl;
    // Computing hash of the pattern
    lli pattern_hash = 0;
    for (lli i = 0; i < P; i++) {
        pattern_hash = (pattern_hash + (pattern[i] - 'a' +
        1) * powers[i]) % M;
    }
    // cout << pattern_hash << endl;
    // Counting occurrences of the pattern in the text
    lli count = 0;
    for (lli i = 0; i + P - 1 < T; i++) {
        lli current_hash = (hash[i + P] + M - hash[i]) %
        M;
        // cout << current_hash << endl;
        if (current_hash == pattern_hash * powers[i] % M)
        {
            count++;
        }
    }
    return count;
}

void soln() {
    string text, pattern;
    cin >> text >> pattern;
    lli ans = RK(text, pattern);
    cout << ans << endl;
}

```

TopologicalSorting.cpp

```

// Using Kahn's algorithm
vector<lli> topologicalSort(vector<vector<lli>> &lis, lli
v) {
    vector<lli> inDegree(v, 0);
    for (lli i = 0; i < v; i++) {
        for (auto j : lis[i]) {
            inDegree[j]++;
        }
    }
    queue<lli> q;
    for (lli i = 0; i < v; i++) {
        if (inDegree[i] == 0) {
            q.push(i);
        }
    }
    vector<lli> ans;
    while (!q.empty()) {
        lli node = q.front();
        q.pop();
        ans.push_back(node);
        for (auto j : lis[node]) {
            inDegree[j]--;
            if (inDegree[j] == 0) {
                q.push(j);
            }
        }
    }
    if ((lli)ans.size() != v) {
        return {};
    }
    return ans;
}

void soln() {

```

```

lli n, m;
cin >> n >> m;
vector<vector<lli>> lis(n);
for (lli i = 0; i < m; i++) {
    lli u, v;
    cin >> u >> v;
    lis[u].push_back(v);
}
vector<lli> ans = topologicalSort(lis, n);
if (ans.empty()) {
    // Graph Contains Cycle, cannot be sorted
    cout << "[ ]" << endl;
    return;
}
for (auto i : ans) {
    cout << i << " ";
}
cout << endl;
}

```

Trie1.cpp

```

struct Node {
    Node* links[26];
    bool flag = false;

    bool containsKey(char ch) { return (links[ch - 'a']) != NULL; }
    void put(char ch, Node* node) { links[ch - 'a'] = node; }
    Node* get(char ch) { return links[ch - 'a']; }
    void endMark() { flag = true; }
    bool isEnd() { return flag; }
};

class Trie {
private:
    Node* root;

public:
    Trie() { root = new Node(); }

    void insert(string word) {
        Node* node = root;
        for (int i = 0; i < (int)word.length(); i++) {
            if (!node->containsKey(word[i])) {
                node->put(word[i], new Node());
            }
            node = node->get(word[i]);
        }
        node->endMark();
    }

    bool search(string word) {
        Node* node = root;
        for (int i = 0; i < (int)word.length(); i++) {
            if (!node->containsKey(word[i])) {
                return false;
            }
            node = node->get(word[i]);
        }
        return node->isEnd();
    }

    bool startsWith(string prefix) {
        Node* node = root;
        for (int i = 0; i < (int)prefix.length(); i++) {
            if (!node->containsKey(prefix[i])) {
                return false;
            }
            node = node->get(prefix[i]);
        }
        return true;
    }

    void display(Node* curr, string prefix) {
        if (curr->isEnd()) {
            cout << prefix << " ";
        }
        for (int i = 0; i < 26; i++) {
            if (curr->links[i] != NULL) {
                char nextChar = i + 'a';
                display(curr->links[i], prefix +
nextChar);
            }
        }
    }

    void displayAll() { display(root, ""); }
};

```